difference is that the neural network *calculates* the value at each location (address), rather than the impossibly large task of *storing* each value. In fact, neural network architectures are often evaluated by how well they separate the hyperspace for a given number of weights.

This approach also provides a clue to the number of nodes required in the hidden layer. A parameter space of N dimensions requires N numbers to specify a location. Identifying a *region* in the hyperspace requires $2N$ values (i.e., a minimum and maximum value along each axis defines a hyperspace rectangular solid). For instance, these simple calculations would indicate that a neural network with 1000 inputs needs 2000 weights to identify one region of the hyperspace from another. In a fully interconnected network, this would require two hidden nodes. The number of regions needed depends on the particular problem, but can be expected to be far less than the number of dimensions in the parameter space. While this is only a crude approximation, it generally explains why most neural networks can operate with a hidden layer of 2% to 30% the size of the input layer.

A completely different way of understanding neural networks uses the DSP concept of *correlation*. As discussed in Chapter 7, correlation is the optimal way of detecting if a known pattern is contained within a signal. It is carried out by multiplying the signal with the pattern being looked for, and adding the products. The higher the sum, the more the signal resembles the pattern. Now, examine Fig. 26-5 and think of each hidden node as looking for a specific pattern in the input data. That is, each of the hidden nodes *correlates* the input data with the set of weights associated with that hidden node. If the pattern is present, the sum passed to the sigmoid will be large, otherwise it will be small.

The action of the sigmoid is quite interesting in this viewpoint. Look back at Fig. 26-1d and notice that the probability curve separating two bell shaped distributions resembles a sigmoid. If we were manually designing a neural network, we could make the output of each hidden node be the *fractional probability* that a specific pattern is present in the input data. The output layer repeats this operation, making the entire three-layer structure a correlation of correlations, a network that looks for *patterns of patterns*.

Conventional DSP is based on two techniques, *convolution* and *Fourier analysis*. It is reassuring that neural networks can carry out both these operations, plus *much more*. Imagine an N sample signal being filtered to produce another N sample signal. According to the output side view of convolution, each sample in the output signal is a weighted sum of samples from the input. Now, imagine a two-layer neural network with N nodes in each layer. The value produced by each output layer node is also a weighted sum of the input values. If each output layer node uses the same weights as all the other output nodes, the network will implement linear convolution. Likewise, the DFT can be calculated with a two layer neural network with N nodes in each layer. Each output layer node finds the amplitude of one frequency component. This is done by making the weights of each output layer node the same as the sinusoid being looked for. The resulting network correlates the